ЛЕКЦИЯ 5 · ИНСТРУМЕНТЫ РАЗРАБОТКИ

Сеть: SSH, порты, HTTP, WebSocket, gRPC

Как сервисы общаются по сети: от TCP/IP и SSH до прикладных протоколов — HTTP, WebSocket и gRPC, и когда что применять

Чернов Евгений
Магистратура «Искусственный интеллект», НИУ ВШЭ

О ЧЁМ ЛЕКЦИЯ

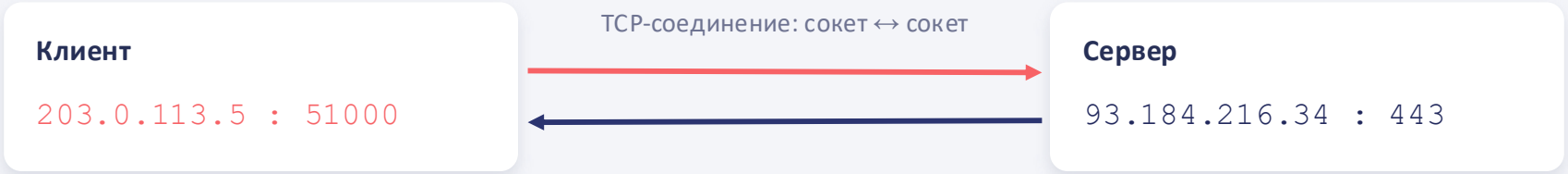
План занятия

1. **TCP/IP и порты**
   IP-адреса, порты, TCP vs UDP, сокеты
2. **SSH**
   ключи, вход, проброс портов, scp/rsync
3. **HTTP**
   запрос/ответ, методы, статусы, REST
4. **Real-time**
   polling, long polling, WebSocket
5. **gRPC**
   protobuf, HTTP/2, стриминг, vs REST
6. **Порты и контейнеры**
   мостик к Docker

СЕТЕВАЯ БАЗА

Модель TCP/IP: адрес, порт, сокет

Данные идут между хостами по IP-адресу; порт выбирает конкретную программу на хосте. Пара «IP + порт» — это сокет, конечная точка соединения.



TCP — с установкой соединения, надёжно, по порядку

HTTP, SSH, gRPC — где важна целостность

UDP — без соединения, без гарантий, зато быстро

DNS, видеозвонки, игры — где важна скорость

АНАЛОГИЯ

Порт — как номер квартиры

Чтобы письмо дошло, адреса дома мало — нужен ещё номер квартиры. В сети так же: IP находит компьютер, а порт — конкретную программу на нём.

Дом (хост) `93.184.216.34`

`:22`	SSH — вход в терминал
`:80`	HTTP — веб-сервер
`:443`	HTTPS — веб по TLS
`:5432`	PostgreSQL — база

«Порт занят» = в эту квартиру уже въехал другой сервис.

`IP-адрес` = адрес дома
находит компьютер в сети

`Порт` = номер квартиры
находит программу на нём (0–65535)

`Сокет` = IP + порт
конкретная дверь, к которой стучимся

01

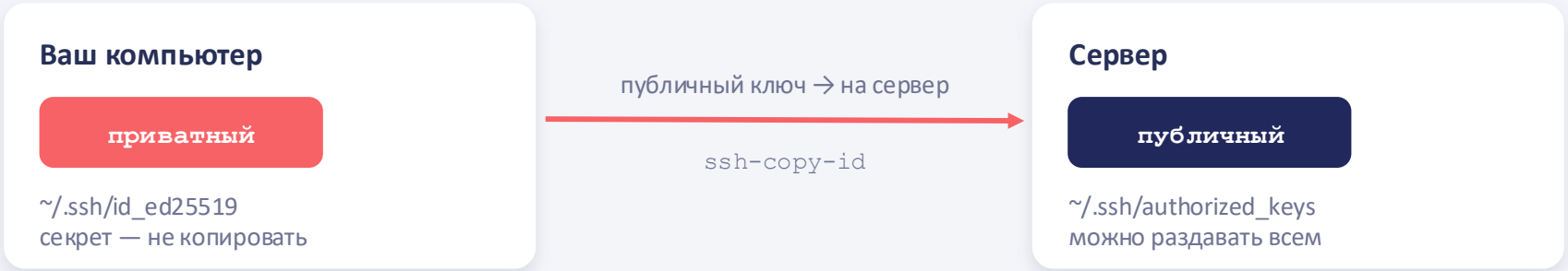
РАЗДЕЛ

SSH: безопасный доступ к серверу

Асимметричная криптография, ключи, вход без пароля и проброс портов

Асимметричная криптография: пара ключей

SSH не шлёт пароль по сети. У вас пара ключей: приватный хранится только у вас, публичный кладётся на сервер. Что подписано одним — проверяется другим.



Вход: сервер даёт задачу, решить которую можно только приватным ключом. Вы доказываете, кто вы, **не пересылая секрет**.

SSH

Генерация ключа и вход без пароля

терминал

```
$ ssh-keygen -t ed25519 -C you@hse
Generating public/private ed25519 pair
Saved: ~/.ssh/id_ed25519     приватный
Saved: ~/.ssh/id_ed25519.pub публичный

$ ssh-copy-id you@server
# ключ дописан в authorized_keys

$ ssh you@server
Welcome — вошли без пароля
```

`ssh-keygen`
создать пару ключей (ed25519)

`id_ed25519.pub`
публичный — уходит на сервер

`ssh-copy-id`
добавить ключ в authorized_keys

`authorized_keys`
доверенные ключи на сервере

`ssh you@server`
вход по подписи, без пароля

Проброс портов: local и remote forwarding

Проброс гонит TCP-порт через зашифрованный SSH-туннель — удалённый сервис становится доступен так, будто он локальный.

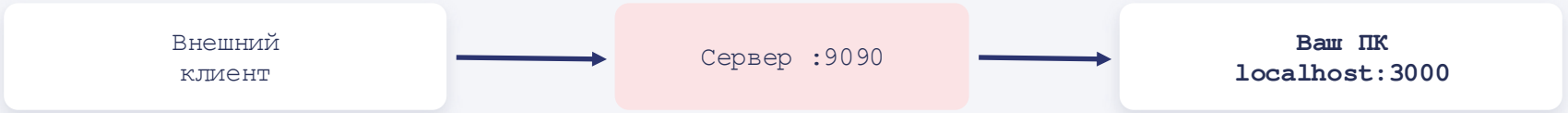
-L Local: локальный порт → в сеть сервера

```
ssh -L 8080:localhost:80 srv
```



-R Remote: порт на сервере → в вашу сеть

```
ssh -R 9090:localhost:3000 srv
```



Левое число — где открываем порт, после двоеточия — куда он ведёт.

Проброс портов: зачем это на практике

Боевая база на сервере слушает только localhost:5432 — снаружи порт закрыт фаерволом. Как подключить к ней локальный psql, ничего не открывая в интернет?

терминал

```
# на сервере БД слушает только localhost
$ ssh -L 5432:localhost:5432 you@srv
# туннель поднят: локальный :5432 → БД

$ psql -h localhost -p 5432
psql (16.2) — подключились к боевой БД
db=> select count(*) from users;
 12480
```

Безопасно
порт БД не торчит в интернет; доступ только по SSH-ключу

Удобно
локальные GUI и psql работают как с локальной базой

Гибко
так же ходят к Redis, админкам и дашбордам за фаерволом

SSH

Копирование файлов: scp и rsync

терминал

```
$ scp file.txt you@srv:/opt/app/
file.txt                100%  4KB
$ scp -r build/ you@srv:/opt/app/
# -r — рекурсивно, всю папку

$ rsync -avz data/ you@srv:/data/
sending incremental file list
sent 2.1M  speedup is 12.4x
# передал только изменённое
```

scp

Просто и всегда целиком

копирует поверх SSH; повторный запуск шлёт всё заново

rsync

Умно и инкрементально

шлёт только различия; -z сжимает, умеет докачку

02

РАЗДЕЛ

Протоколы прикладного уровня

HTTP, WebSocket и gRPC — как приложения договариваются поверх TCP

HTTP: запрос и ответ

> запрос · < ответ

```
$ curl -v https://api.site/users/1
> GET /users/1 HTTP/1.1
> Host: api.site
> Accept: application/json

< HTTP/1.1 200 OK
< Content-Type: application/json

{"id":1,"name":"Ann"}
```

Методы

GET	прочитать
POST	создать
PUT/PATCH	изменить
DELETE	удалить

Классы статусов

2xx	успех
3xx	редирект
4xx	ошибка клиента
5xx	ошибка сервера

Анатомия запроса и ответа: строка за строкой

Запрос и ответ устроены одинаково: стартовая строка → заголовки → пустая строка → тело.

Запрос — шлёт клиент

1 · стартовая строка

```
GET /users/1 HTTP/1.1
```

метод · ресурс · версия

2 · заголовки

```
Host: api.site
Accept: application/json
```

3 · пустая строка

сигнал «заголовки кончились»

4 · тело

у GET нет; у POST — JSON/форма

Ответ — шлёт сервер

1 · статусная строка

```
HTTP/1.1 200 OK
```

версия · код · пояснение

2 · заголовки

```
Content-Type: application/json
Content-Length: 21
```

3 · пустая строка

и сразу за ней — тело

4 · тело

```
{"id":1,"name":"Ann"}
```

Один запрос «на проводе»: от DNS до ответа

curl -v: * связь · > запрос · < ответ

```
$ curl -v https://api.site/users/1
* Trying 93.184.216.34:443...
* Connected to api.site :443
* TLS handshake, TLSv1.3
> GET /users/1 HTTP/2
> host: api.site
> accept: */*
< HTTP/2 200
< content-type: application/json
{"id":1,"name":"Ann"}
* Connection #0 kept alive
```

- `*` **DNS** — имя → 93.184.216.34
- `*` **TCP** — рукопожатие, порт :443
- `*` **TLS** — шифруем канал (https)
- `>` **Запрос** — метод, путь, заголовки
- `<` **Ответ** — статус, заголовки, тело
- `*` **Keep-alive** — соединение не рвём

REST-семантика и HTTP/1.1 vs HTTP/2

REST — соглашение: URL адресует ресурс, а HTTP-метод задаёт действие. Один URL + разные методы = разные операции.

URL = ресурс, метод = действие

```
GET    /users      список
POST   /users      создать
GET    /users/42   один
PUT    /users/42   заменить
DELETE /users/42   удалить
```

HTTP/1.1 текстовый протокол

один запрос за раз в соединении; на страницу открывают несколько TCP-соединений

HTTP/2 бинарный протокол

мультиплексирование: много потоков в одном соединении; сжатие заголовков (HPACK)

REAL-TIME

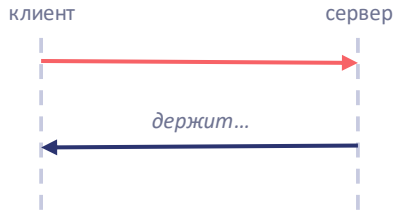
Три модели обновлений в реальном времени

Short polling



клиент спрашивает по таймеру снова и снова — часто впустую

Long polling



сервер держит запрос открытым, пока не появятся данные

WebSocket



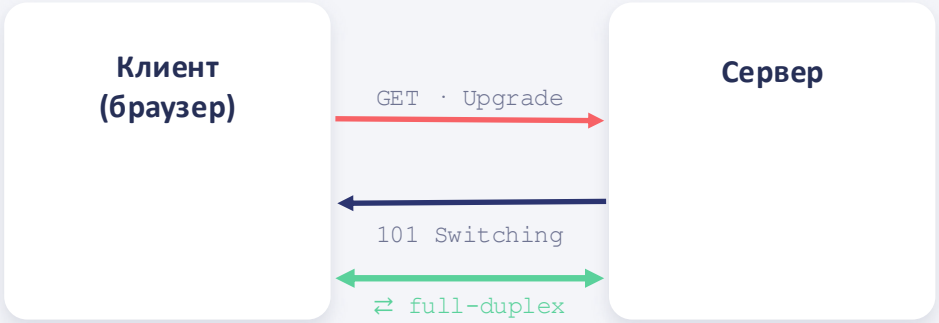
одно соединение — оба шлют кадры в любой момент

Слева направо — всё «живее» и меньше лишнего трафика.

REAL-TIME

WebSocket: полнодуплексный канал

WebSocket начинается как обычный HTTP-запрос с заголовком Upgrade. Затем то же TCP-соединение превращается в двусторонний канал, живущий сколько угодно.



Когда применять

чаты и уведомления, live-дашборды и котировки, онлайн-игры, совместное редактирование

Когда достаточно HTTP

редкие обновления, обычный запрос-ответ, простой CRUD

REAL-TIME

Polling против WebSocket: что выбрать

	Polling (HTTP)	WebSocket
Модель	клиент сам спрашивает	сервер сам присылает
Задержка	до интервала опроса	мгновенный пуш
Трафик	много пустых ответов	один канал, без лишнего
Сервер	stateless, просто	держит открытые соединения
Масштаб	как обычный HTTP	тысячи живых сокетов
Когда брать	редкие обновления	частые, двусторонние

gRPC: контракт в .proto

user.proto

```
syntax = "proto3";

service UserService {
  rpc GetUser (Id) returns (User);
  rpc List (Empty)
      returns (stream User);
}
message User {
  int32  id   = 1;
  string name = 2;
}
```

protobuf

строгая схема, бинарь — компактно

RPC

удалённый вызов как локальный

HTTP/2

мультиплексирование, один канал

stream

потоки: сервер / клиент / оба

gRPC против REST

	REST	gRPC
Формат данных	JSON — текст	Protobuf — бинарь
Контракт	OpenAPI (опционально)	.proto (обязателен)
Транспорт	HTTP/1.1 и выше	HTTP/2
Стриминг	ограниченный	встроенный, 2-сторонний
Читаемость	человекочитаемо	нужен декодер
Сильная сторона	публичные API, браузер	межсервисное, низкая задержка

Когда gRPC, а когда REST

Это не «или-или»: у каждого своя зона. Ориентируйтесь на клиента и на границу системы.

REST берите, когда…

- API для внешних клиентов и партнёров
- запросы прямо из браузера / фронтенда
- нужен читаемый JSON и отладка через curl
- CRUD над ресурсами, кэширование ответов

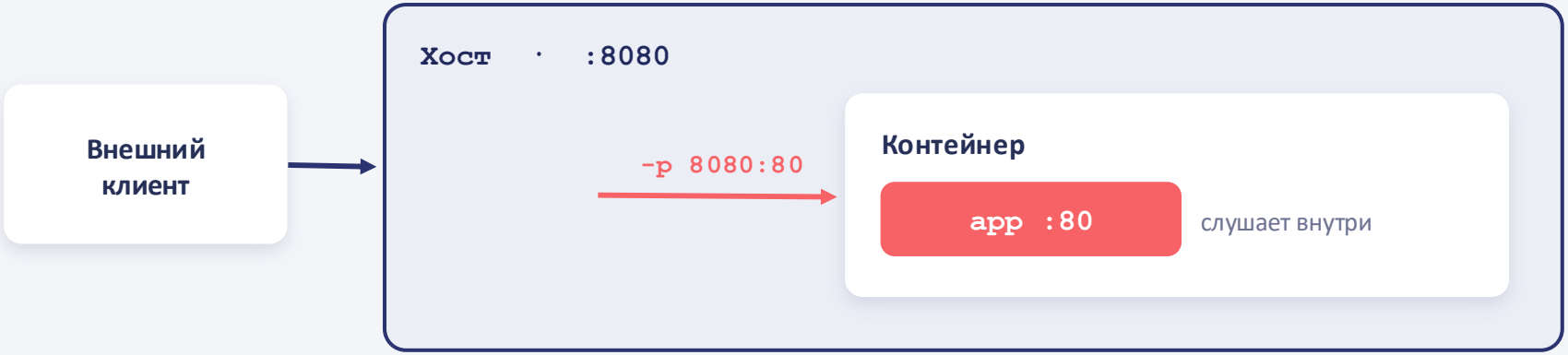
gRPC берите, когда…

- общение между своими микросервисами
- важны низкая задержка и компактность
- нужен строгий контракт и кодогенерация
- потоки данных в обе стороны (streaming)

! **На практике — вместе:** gRPC между внутренними сервисами, REST/JSON наружу для браузера и партнёров.

Порты в контейнерах — мостик к Docker

Контейнер (из темы 5) имеет свой сетевой namespace: собственный localhost и свои порты. Чтобы сервис был доступен снаружи, порт публикуют на хост.



`docker run -p 8080:80 img` — снаружи :8080 попадает в контейнерный :80. Тот же принцип, что SSH-проброс.

ИТОГИ ЛЕКЦИИ

Что важно унести

1. **HTTP / REST**
   запрос-ответ: публичные API, CRUD, браузер — основа веба

2. **WebSocket**
   двусторонний real-time: чаты, уведомления, live-дашборды

3. **gRPC**
   межсервисное общение: строгий контракт, HTTP/2, низкая задержка

4. **SSH и порты**
   доступ по ключам и туннели; порт — это адрес сервиса